

Decision Tree

Assignment

Dataset Link : Bank_Marketing

Submission Guidelines

Students must submit:

- Code (Colab / Jupyter Notebook)
- Outputs (results)
- Proper interpretation for each step

Section 1: Theoretical Questions

Q1. What is a Decision Tree? Explain its structure (Root, Nodes, Leaves) with a real-life example.

Answer:

A Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It works like a flowchart where data is split into branches based on conditions.

Structure of Decision Tree

1. Root Node

- The top-most node.
- Represents the main feature used for the first split.

2. Decision Nodes

- Intermediate nodes where further splitting occurs.

3. Leaf Nodes

- Final output or prediction.
- No further splitting occurs.

Real-Life Example

Suppose a bank wants to predict whether a customer will purchase a fixed deposit.

Example Decision Flow:

- **Root Node: Does customer have a housing loan?**
 - Yes → Check income
 - No → Check age
- **Leaf Node:**
 - “Yes” → Customer likely to subscribe
 - “No” → Customer unlikely to subscribe

Decision Trees mimic human decision-making processes.

Q2. Differentiate between Gini Impurity and Entropy. Which one is used by default in Scikit-learn and why?

Answer:

Basis	Gini Impurity	Entropy
Formula	Measures impurity using probability	Measures impurity using information gain
Speed	Faster	Slower
Complexity	Simpler calculation	More computationally expensive
Range	0 to 0.5	0 to 1

Default in Scikit-learn

Scikit-learn uses Gini Impurity by default.

Reason:

- Faster computation
- Efficient for large datasets
- Produces similar results to Entropy

Q3. What is Overfitting in Decision Trees? How can we detect it using training and testing accuracy?

Answer:

Overfitting occurs when the model learns training data too well, including noise and outliers, resulting in poor performance on unseen data.

Detection using Accuracy

Scenario	Training Accuracy	Testing Accuracy
Good Fit	High	High
Overfitting	Very High	Low
Underfitting	Low	Low

Example:

- Training Accuracy = 99%
- Testing Accuracy = 70%

This indicates overfitting.

Q4. Explain Pruning in Decision Trees. What is the difference between Pre-pruning and Post-pruning?

Answer:

Pruning is the process of reducing unnecessary branches in a Decision Tree to prevent overfitting.

Types of Pruning

Pre-pruning	Post-pruning
Stops tree growth early	Removes branches after full growth

Faster

More accurate

Uses parameters like
max_depth

Uses validation performance

Pre-pruning Parameters:

- max_depth
- min_samples_split
- min_samples_leaf

Q5. What is Feature Importance? How can it help businesses in decision-making?

Answer:

Feature Importance measures how much each feature contributes to the prediction.

Business Benefits

- Identifies key customer behavior patterns
- Helps focus on important business variables
- Improves marketing strategies
- Reduces unnecessary data collection

Example:

If “duration” is highly important in bank marketing:

- Banks can improve customer call interaction quality.

Section 2: Practical Questions (Hands-on using your dataset)

Q6. Data Understanding

- Load the dataset
- Display first 5 rows
- Check:
- Shape

- Data types
- Column names

Interpret:

- **What types of variables are present?**

Answer:

```
# Import libraries
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("bank.csv")

# Display first 5 rows
print(df.head())

# Shape
print("Shape:", df.shape)

# Data types
print(df.dtypes)

# Column names
print(df.columns)
```

Interpretation:

- **Dataset contains both:**
 - Numerical variables
 - Categorical variables

Examples:

Numerical:

- age
- balance
- duration

Categorical:

- job
- marital
- education

Q7. Data Cleaning

- Check for missing values
- Handle missing values (if any)
- Check for duplicates

Interpret:

- **Is the dataset clean? What issues did you find?**

Answer:

```
# Missing values
print(df.isnull().sum())

# Handle missing values if any
df.fillna(method='ffill', inplace=True)

# Check duplicates
print("Duplicate rows:", df.duplicated().sum())

# Remove duplicates
df.drop_duplicates(inplace=True)

print("Dataset cleaned successfully")
```

Interpretation:

- **Dataset may contain:**
 - Missing values
 - Duplicate rows

After cleaning:

- Dataset becomes more reliable for modeling.

Q8. Data Preprocessing

- Convert categorical variables into numerical
- Mention which encoding method you used

Interpret:

- **Why is encoding necessary for Decision Trees?**

Answer:

```
# Convert categorical columns into numerical  
df_encoded = pd.get_dummies(df, drop_first=True)
```

```
print(df_encoded.head())
```

Encoding Method Used:

- One-Hot Encoding

Interpretation:

Decision Trees require numerical input because:

- ML algorithms cannot directly process text/categorical values.
- Encoding converts categories into machine-readable format.

Q9. Feature Selection & Splitting**Separate:**

- Features (X)

- Target (y)
- Perform train-test split (80-20)

Interpret:

- **Why is train-test split important?**

Answer:

```
from sklearn.model_selection import train_test_split
```

```
# Features
```

```
X = df_encoded.drop('y_yes', axis=1)
```

```
# Target
```

```
y = df_encoded['y_yes']
```

```
# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
print(X_train.shape)
```

```
print(X_test.shape)
```

Interpretation:

Train-test split is important because:

- Training data teaches the model.
- Testing data evaluates performance on unseen data.
- Prevents biased evaluation.

Q10. Model Building

- Train a Decision Tree Classifier

Use:

- criterion='gini'
- max_depth=5

Interpret:

- What does max_depth control?

Answer:

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Create model
```

```
dt_model = DecisionTreeClassifier(  
    criterion='gini',  
    max_depth=5,  
    random_state=42  
)
```

```
# Train model
```

```
dt_model.fit(X_train, y_train)
```

```
print("Model trained successfully")
```

Interpretation:

max_depth controls:

- Maximum number of levels in the tree.
- Smaller depth prevents overfitting.
- Larger depth increases model complexity.

Q11. Model Evaluation

Perform:

- Accuracy score
- Confusion Matrix
- Classification Report

Interpret (MUST):

- Is the model performing well?
- Explain Precision & Recall
- Any class imbalance issue?

Answer:

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
# Predictions
```

```
y_pred = dt_model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:")
```

```
print(cm)
```

```
# Classification Report
```

```
print(classification_report(y_test, y_pred))
```

Interpretation

Is model performing well?

- If accuracy is high (e.g., above 80%), model performs reasonably well.

Precision

- Out of predicted “yes”, how many were actually “yes”.

Recall

- Out of actual “yes”, how many were correctly predicted.

Class Imbalance

If one class has much higher records:

- Accuracy alone may become misleading.
- Precision and Recall become important.

Q12. Overfitting Check

- **Calculate:**
 - Training accuracy
 - Testing accuracy

Interpret:

- **Is the model overfitting or underfitting?**

Answer:

```
# Training accuracy
train_acc = dt_model.score(X_train, y_train)

# Testing accuracy
test_acc = dt_model.score(X_test, y_test)

print("Training Accuracy:", train_acc)
print("Testing Accuracy:", test_acc)
```

Interpretation:

- If training accuracy >> testing accuracy:
 - Overfitting exists.
- If both accuracies are low:
 - Underfitting exists.
- If both are similar and high:
 - Model is balanced.

Q13. Pruning Experiment

-
- **Train another model with:**
- max_depth=3
- min_samples_split=20

Interpret:

- **Compare performance with previous model**
- **Which one is better and why?**

Answer:

```
# Pruned model
pruned_model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    min_samples_split=20,
    random_state=42
)

# Train model
pruned_model.fit(X_train, y_train)

# Accuracy
pruned_train_acc = pruned_model.score(X_train, y_train)
pruned_test_acc = pruned_model.score(X_test, y_test)

print("Pruned Training Accuracy:", pruned_train_acc)
print("Pruned Testing Accuracy:", pruned_test_acc)
```

Interpretation:

Comparison:

- Original model:
 - More complex
 - Higher overfitting risk
- Pruned model:
 - Simpler

- Better generalization

Better Model:

The better model is the one with:

- Balanced training and testing accuracy
- Lower overfitting

Q14. Feature Importance

- Extract top 5 important features
- Plot feature importance graph

Interpret:

- **Which features influence prediction the most?**

Answer:

```
import matplotlib.pyplot as plt

# Feature importance
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
})

# Top 5 features
top5 = importance.sort_values(
    by='Importance',
    ascending=False
).head(5)

print(top5)

# Plot
plt.figure(figsize=(8,5))

plt.bar(top5['Feature'], top5['Importance'])
```

```
plt.xlabel("Features")
plt.ylabel("Importance")

plt.title("Top 5 Important Features")

plt.show()
```

Interpretation:

The most important features highly influence customer subscription prediction.

Common Important Features:

- duration
- balance
- age
- campaign
- housing loan

Q15. Business Insights

Based on your model:

Answer:

- Which type of customers are more likely to say “yes”?
- What strategy should a bank us

Answer:

Which customers are more likely to say “yes”?

Customers are more likely to subscribe if they:

- Have higher account balance
- Spend longer time during calls

- Have no housing loan
- Are middle-aged professionals

What strategy should the bank use?

The bank should:

- Target customers with high balances
- Improve telemarketing call quality
- Focus campaigns on customers with higher response probability
- Use personalized marketing strategies

This can improve:

- Conversion rates
- Marketing efficiency
- Customer engagement